Virtual KAM: submission note

Pashmeen Kaur · Snapdeal AI First Intern take-home Live demo: https://snapdeal-kam-chi.vercel.app · Code: https://github.com/10apashmeenkaur-boop/snapdeal-kam

What it does

Asha is a voice AI key account manager for a seller whose sales dropped. She doesn't just talk about fixes: she **diagnoses the drop from data, asks only what the data can't tell her, and executes the fixes during the call**, after the seller says yes. The seller dashboard changes live as she works. Every call ends with a saved summary and a 7-day follow-up with measurable targets. For the brief's seller she finds: sales −₹65,000 (−32.5%), all volume since price was flat; 13 products newly out of stock cost about 97 orders (75% of the drop), 4 bestsellers alone 88; cancellations rose 8% → 14%, mostly "no stock". Two more mock sellers have different root causes (pricing vs a competitor, slow dispatch), so the diagnosis is computed, not scripted.

Architecture

- **ElevenLabs Agents** runs the voice loop: speech-to-text, LLM, text-to-speech, turn-taking and barge-in. Connects over WebRTC, with a WebSocket fallback.
- **Client tools.** The agent's 13 tools run in the browser: 5 read, 8 write. Each calls the app's API and flashes the change on the dashboard. No public webhook needed.
- **Next.js API.** `/api/tools/<name>` validates arguments with zod and enforces guardrails *in code*: write tools refuse without `seller_confirmed: true`; deals are capped at 30% and can't price below cost + 5%. `/api/get-signed-url` mints short-lived call credentials, so the API key never reaches the browser and the agent stays private.
- **Deterministic analytics.** All maths (volume vs price split, stock-out impact, excess cancellations, residual) lives in a unit-tested TypeScript engine; the LLM only repeats the numbers it gets back.
- **Upstash Redis** keeps one shared state across serverless instances (local dev uses a JSON file).
- **Config as code.** Prompt, tools and settings in `agent/` are pushed by `npm run agent:sync`; a test fails if tool names drift between app and agent.

Models and voice

LLM **Gemini 3.5 Flash** (temperature 0.4). Speech-to-text: ElevenLabs ASR, English by default, switching to Hindi via the `language_detection` tool. Text-to-speech: ElevenLabs **Flash v2** (English) / **Flash v2.5** (Hindi), voice **"Anika"** (Indian English, customer care, handles Hinglish). Calls are capped at 4 minutes.

One turn, end to end

Seller says "*Haan, pause kar do*" → ASR → Gemini decides `mark_listings_unavailable(skus, seller_confirmed: true)` → the browser runs the client tool → `POST /api/tools/…` → zod + consent check → store updated → short reply ("Done. Marked 4 listings unavailable…") → dashboard rows flash → Gemini phrases one or two sentences → Flash TTS speaks. Across a call: check history and diagnosis silently → one-sentence headline → at most 3 questions → 2–3 actions with numbers (Asha's vs the seller's) → act on "yes" → schedule follow-up → save summary → `end_call`. Angry sellers and refund or exception requests go to a human KAM.

Assumptions

Seller profiles, SKU data, cancellation reasons, competitor prices, sellers S2/S3 and the "Simulate 7 days" effects are mock. Orders = orders placed; Sales = Orders × ASP (matches the brief). Actions change a mock backend; in production they'd call Snapdeal seller-panel APIs.

What I'd build next

- **Follow-up memory:** the 7-day call opens by reviewing last week's agreed actions against new numbers. The call logs, `get_account_history` and the week simulation already exist.
- **Proactive calls:** the anomaly health check (e.g. "4 bestsellers just went out of stock") triggers Asha to call the seller first.
- Real phone calls via Exotel/Twilio, real seller-panel APIs, more Indian languages, a human-KAM handoff queue.
- Call evaluation (did the seller act, how much sales recovered), per-call cost tracking, DPDP-compliant recording consent.